INTERNAL PROJECT UPDATE

Warehouse Sack Counting — Approach Review

A comparison of the computer-vision approaches tested so far for automatically counting stacked rice bags from a warehouse photo, and the reasoning behind the current direction.

Prepared for: President **Subject:** Rice Bag / Warehouse Sack Counting POC

Date: September 2026

We have tested four working prototypes. Two of them turned out to be built on the exact same underlying detection engine, so they are best understood as one approach with an added feature, not two separate methods. Below is what each one actually does, where it holds up, and where it doesn't.

The four prototypes

TRIED FIRST · TWO TOOLS, ONE ENGINE ON HOLD

Detection & Duplicate Cleanup

`test_rice_bag_v9_ui.py` + `warehouse_pile_estimator_ui.py`

WHAT IT IS

A trained model draws a box around every rice bag it sees in a photo. We confirmed by reading both files that `warehouse_pile_estimator_ui.py` loads the identical trained model and the identical duplicate-cleanup logic as `test_rice_bag_v9_ui.py` — it is the same detection engine,

with a pile-measuring step bolted on top. They should be counted as one approach.

HOW IT COUNTS

Because bags are packed tightly and have repeating fold and shadow lines, the raw model often draws two or three boxes on one physical bag. A two-step cleanup runs after every detection: first it merges boxes that clearly overlap or sit inside one another; then it merges any remaining duplicates whose centers are still suspiciously close together for the size of bag in that photo. What survives is marked as one dot per bag, numbered, with a running total.

VOLUME STEP

`warehouse_pile_estimator_ui.py` adds one thing the other tool doesn't have: a person draws a reference line across the pile's length, breadth, and height directly on the photo, and the tool counts how many already-detected bags fall along each line, then multiplies the three counts to estimate total bags in the pile.

STRENGTHS

Fast, already produces a usable count from a live photo, and it's the only prototype that currently attempts any length/breadth/height estimate at all.

LIMITS

Still fundamentally a box-around-each-bag approach, so two touching bags can still be miscounted as one or one bag double-counted in the densest piles. The cleanup thresholds are hand-tuned. The length/breadth/height step needs a person to draw the lines each time.

TRIED SECOND SUPERSEDED

Sack “Keypoint” Detection

`test_sack_keypoint_ui.py`

WHAT IT IS

A second detection model, trained on a smaller, separate set of warehouse photos, running through the same detect-then-clean-up pipeline as the first approach. Despite the name, it is a box-detection model, not a true keypoint model.

STRENGTHS

Proved the same duplicate-cleanup approach carries over to a different model and dataset.

LIMITS

Trained on notably fewer example photos than the first approach. An early version of this tool also had a bug that silently capped every photo's count at exactly 300 bags regardless of the real number — since fixed, but it is a reminder that these counts need spot-checking, not blind trust. Same box-based overlap risk as the first approach.

CURRENT FOCUS ACTIVE DEVELOPMENT

Instance Segmentation

`test_rice_bag_v11_segmentation_ui.py`

WHAT IT IS

A different kind of model entirely. Instead of learning to draw a rectangle around each bag, it learns the actual visible outline of every bag — trained on photos where each bag was hand-traced, not boxed. It reports three categories: individual bags, pile, and other objects.

HOW IT COUNTS

Every detected bag gets its own traced boundary, a fill color, a center point, and a label, drawn directly on the photo. Because each bag's true shape is captured rather than a rectangle guess, two bags that are touching or partly stacked are told apart by their own outlines instead of by boxes that have to be untangled afterward.

STRENGTHS

The only approach where overlapping and tightly stacked bags are distinguished by their real shape. Also separately tracks “pile” as its own category — a natural building block for a future volume estimate.

LIMITS

The newest and least mature prototype. Trained on the smallest set of example photos so far, and still being tuned. Has no length/breadth/height step yet. The “other” and “pile” categories in particular have very few training examples and need more.

CURRENT CONCLUSION

Segmentation is the direction we're betting on

Because it identifies each bag individually and produces a separate outline for every one, the segmentation approach has the strongest potential to handle overlapping and tightly stacked bags accurately — the single biggest source of counting error in a real warehouse pile. For that reason, effort right now is going into improving the segmentation model rather than finishing off the earlier detection-based tools.

Next planned step

Once the segmentation model is strong enough, the length/breadth/height line-counting method already built and proven in `warehouse_pile_estimator_ui.py` will be reused as the basis for a separate volume-estimation module — feeding the pile dimensions it captures into a calculation of total warehouse pile volume.

Prepared from a direct code review of all four prototypes

No performance figures are quoted — all four are still early-stage